

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI
WORK INTEGRATED LEARNING PROGRAMMES DIVISION

Deep Reinforcement Learning
Lab Assignment 1

Total Marks = 10 Marks (5M for the first part and 5M for the second part)

Intended Learning Outcome:

Students should be able to

- Understand the basic functionality of Multi-Armed Bandit and Dynamic Programming.
- Implement the concepts of Multi-Armed Bandit and Dynamic Programming.

Prerequisite:

- (1) Students should go through the lectures CS1, CS2, CS3, CS4, CS5;
- (2) Webinar demonstrations

Please note that this assignment will involve some amount of self-learning (on the part of modelling solutions appropriately + programming skills)

Submission Deadline: **th May, 2026**

Instructions:

- Read the assignment proposal carefully.
- Solve both assignment problems. Submit two different solution files. (Team # - MAB, Team # - DP)
- It is mandatory to submit the assignment in **PDF format**, consisting of all the codes and outputs with each and every iteration. Any other format will not be accepted.
- Only one submission per group is required.
- Add comments and descriptions to every function you are creating or operation you are performing. If not found, then 1 mark will be deducted. There are many assignments that need to be evaluated. By providing the comments and description it will help the evaluator to understand your code quickly and clearly.
- No last moment submissions /email submissions will be accepted. Therefore make sure you will submit the first version of your assignment 2 days before the deadline.
- Assignment must be executed in virtual lab and screenshots with timestamp in virtual lab must be attached in the final submission

How to reach out for any clarifications:

This assignment is administered by

- (1) Subash - subasharun@wilp.bits-pilani.ac.in
- (2) Divya K - divyak@wilp.bits-pilani.ac.in
- (3) Dincy R Arikkat - dincyrarikkat@wilp.bits-pilani.ac.in

Any request for clarification must be addressed through email (official email only) to all the three instructors listed.

Messaging in TEAMS is discouraged. This is to ensure we maintain track of all the transactions. If we find any clarifications to be shared across all the students, we will share this using discussion forums.

Part #1 - MAB
Total Marks = 5 Marks

Title

Adaptive Treatment Recommendation System using Multi-Armed Bandit Learning

Problem Scenario

You are a data scientist working in cancer research in collaboration with a medical institution conducting clinical trials for different treatment protocols. A hospital is evaluating multiple medicines for treating a chronic disease, where patient responses vary due to hidden factors such as age, immunity level, genetic background, and disease severity. Since the most effective treatment is not known at the beginning, the goal is to develop an intelligent recommendation system that learns from patient outcomes over time and progressively identifies the optimal medicine for future patients.

Each medicine is treated as an **arm** in a Multi-Armed Bandit (MAB) problem.

Each group must create a **custom synthetic patient-treatment dataset** based on the **group number**.

1. Dataset Creation Rules

Let group number = **G**.

Use: `random.seed(G)`, `numpy.random.seed(G)` to ensure reproducibility.

1.1 Number of Medicines

The number of available treatment choices for your group is:

$$K = (G \bmod 3) + 5$$

1.2 Hidden Success Probability

Each medicine i , where $i \in \{0, 1, \dots, K - 1\}$, has a hidden probability of successful recovery:

$$P_i = 0.4 + ((G + i) \bmod 6) \times 0.07$$

1.3 Patient Severity and Utility Logic

Generate exactly **1000 patient records** sequentially (patient_id from **0 to 999**). Each patient has an inherent **disease severity score** defined as:

$$Severity = (patient_id \bmod 5) + 1$$

This produces severity values ranging from **1 (mild)** to **5 (critical)**.

When a medicine i is assigned to a patient, the **clinical outcome** is binary:

- **1 (Recovered)**: occurs with probability P_i
- **0 (Not Recovered)**: occurs with probability $1 - P_i$

where P_i is the hidden success probability of medicine i .

To simulate real hospital impact, treatment benefit decreases for highly severe patients.

$$UtilityScore = \text{clinical_outcome} \times \left(1 - \frac{Severity}{10}\right)$$

Interpretation:

- If patient recovers and severity = 1 $\rightarrow$ reward = **0.9**
- If patient recovers and severity = 5 $\rightarrow$ reward = **0.5**
- If patient does not recover $\rightarrow$ reward = **0**

1.4 Dataset Schema

Create a dataset with **1000 rows** containing:

Column	Description
patient_id	patient index (0–999)
severity_score	disease severity (1–5)

During execution of each algorithm, dynamically populate:

Column	Description
assigned_medicine	medicine selected by algorithm
clinical_outcome	binary recovery (0/1)
utility_score	final reward

Important:

- Use clinical_outcome to update bandit statistics.
- Use utility_score to compute cumulative reward.

2. Tasks

Task 1: Dataset Design (1 Marks)

Write code to:

1. Generate your synthetic environment.
2. Display:
 - group number G
 - total medicines K
 - hidden success probabilities of all medicines
3. Print first 10 dataset rows.

Task 2: Immediate Exploitation Strategy (1 Marks)

A hospital administrator suggests: *"Once a treatment appears best, continue prescribing only that treatment for all future patients."*

- Implement a recommendation model following this policy.
- Perform 1000 iterations and evaluate cumulative reward.

Hint:

Test each medicine exactly 10 times initially, then continue using only the currently best-performing one.

Task 3: Controlled Clinical Trial Strategy (1.5 Marks)

Doctors suggest: *"Most patients should receive the current best treatment, but occasionally another treatment should be tested to discover hidden opportunities."*

- Implement this strategy with: 10% intentional exploration.
- Run the simulation for 1000 patients and evaluate cumulative reward.

- Analyze what happens if exploration changes to: 1%, 50%.

Task 4: Confidence-Based Strategy (1 Marks)

A senior physician recommends: *"Treatments with fewer observations should initially be given more chances, but this preference should reduce as evidence grows."*

- Implement the UCB1 algorithm and evaluate cumulative reward.

Task 5: Comparative Analysis (0.5 Marks)

Using Matplotlib, generate a **Cumulative Reward vs. Number of Patients** graph to compare the performance of all implemented strategies over 1000 iterations.

Based on the graph and experimental results, answer the following:

1. Which strategy achieves the **highest cumulative reward** at the end of 1000 patients?
2. Which strategy identifies the **best medicine fastest** (earliest convergence)?
3. Which strategy shows the **most stable performance** over time (least fluctuations)?
4. Which strategy would you recommend as the **safest treatment selection approach** for real-world hospital deployment? Justify your answer briefly.

Write a short comparative summary (3–5 sentences) based on your observations.

Important : Assignment must be executed in virtual lab and screenshots with timestamp in virtual lab must be attached in the final submission. Also, the timestamp of the execution, Virtual Machine ID shall be fetched & printed in the top of the Python notebook.

PART #2 - DP

Total Marks = 5 Marks

Autonomous Drone Rescue Using Dynamic Programming

Problem Statement

Design and implement a reinforcement learning agent using dynamic programming - Value Iteration or Policy Iteration to compute an optimal policy for an autonomous rescue drone operating in a disaster zone. The drone must learn how to rescue stranded civilians while managing battery power, avoiding dangerous zones, and reaching charging stations when necessary. The problem must be modelled as a finite Markov Decision Process (MDP). The register number of the first student in a group (alphabetically sorted) will determine the environment configuration. The student will:

- Implement a custom Drone Rescue environment.
- Use dynamic programming to compute the optimal value function and policy.
- Analyse how state design and reward design affect convergence and rescue efficiency.

Scenario

A rescue drone is deployed in a disaster-hit city after an earthquake. The city is represented as a grid world. The drone must:

- rescue stranded civilians,
- avoid dangerous areas,
- manage battery usage,
- and reach charging stations before battery depletion.

The environment contains:

- Safe zones
- Danger zones
- Charging stations
- Rescue targets
- Wind zones that disturb movement

The drone begins with limited battery power and must maximize total rescue reward.

Environment Description:

Grid Size

- If the student ID ends with 0–4:
 - Use a 5×5 grid.
- If the student ID ends with 5–9:
 - Use a 6×6 grid.

Initial Environment Configuration

The placement of rescue targets, charging stations, danger zones, and blocked cells must depend on the student ID.

If the student ID ends with 0–4:

- Use:
 - 2 rescue targets
 - 1 charging station
 - 3 danger zones
 - 2 blocked cells

If the student ID ends with 5–9:

- Use:
 - 3 rescue targets
 - 2 charging stations
 - 4 danger zones
 - 3 blocked cells

Students must clearly explain:

- the grid configuration,
- symbol placement,
- and starting battery level.

Cell Types

Symbol	Meaning
S	Start position
F	Free/Safe cell
D	Dangerous zone
R	Rescue target
C	Charging station
W	Wind zone
X	Blocked cell / obstacle

Drone Behaviour

The drone:

- starts at S, The starting position (S) must be fixed at the top-left corner of the grid.
- has limited battery,
- loses battery every move,
- gains reward by rescuing civilians,
- and must avoid running out of power.

Actions

At each state, the drone may:

- Move Up
- Move Down
- Move Left
- Move Right
- Hover (remain in the same cell)

Hover Action

- If the drone is on a charging station (C):
 - hovering increases battery by +2 units
 - battery cannot exceed maximum capacity.
- If the drone is not on a charging station:
 - hovering consumes 1 battery unit like a normal move.

Special Environment Rules

1. Battery System

Each action consumes 1 battery unit.

The maximum battery capacity depends on the student ID:

- If the student ID ends with an even digit:
 - maximum battery = 10 units

- If the student ID ends with an odd digit:
 - maximum battery = 15 units

If battery becomes 0: the episode terminates immediately.

2. Charging Stations (C)

When the drone enters a charging station: battery becomes full again.

3. Rescue Targets (R)

When the drone reaches a rescue cell:

- reward is obtained,
- the rescue target disappears,
- the cell becomes a free cell (F).

4. Dangerous Zones (D)

Danger cells represent fire, unstable buildings, or radiation.

Entering a danger zone:

- gives heavy negative reward,
- but does not terminate the episode immediately.

5. Wind Zones (W)

Wind cells introduce stochastic transitions.

If the drone is currently on a wind cell (W) and attempts a movement action: there is a probability that the movement direction changes randomly.

If wind disturbance occurs: the actual movement direction must be selected uniformly from:

- Up,
- Down,
- Left,
- Right.

The wind probability depends on the student ID:

- 20% if the student ID ends with 0–4
- 30% if the student ID ends with 5–9

The student must clearly explain the transition dynamics used.

6. Blocked Cells (X)

Blocked cells represent inaccessible regions such as collapsed buildings or debris.

- The drone cannot enter blocked cells.
- If an action would move the drone into a blocked cell:
 - the drone remains in its current position,
 - the action still consumes 1 battery unit.

Episode Termination

An episode ends when:

- Battery becomes zero
- All rescue targets are rescued
- Maximum step limit exceeded: 50 for 5×5 grids and 75 for 6×6 grids.

Environment Dynamics:

1. State Space

Each state should minimally contain:

- Drone position (row, col)
- Current battery level
- Status of each rescue target (rescued / not rescued)

The student must:

- clearly explain the state representation.

2. Action Space

Possible actions: Up, Down , Left , Right, Hover

The student must implement a function that returns all valid actions available from a given state. The student must ensure that only valid actions are allowed at each state.

3. Rewards

The environment must use the following reward structure.

Event	Reward
Rescue target reached	20
Enter danger zone	-10
Battery exhausted	-20
Reach charging station	5
Regular movement	-1

Expected Outcomes

1. Custom Drone Rescue Environment – 1 mark

Implement a custom environment with: `reset()` , `step(action)` and `render()`. The environment must correctly handle: battery updates, rescue target removal, stochastic wind movement, charging stations, blocked cells and reward computation.

2. Dynamic Programming Solution – 2 mark

Using Value Iteration or Policy Iteration:

1. Enumerate reachable states.
2. Compute:

- optimal value function $V^*(s)$
 - optimal policy $\pi^*(s)$
3. Use stopping threshold:
- $\theta=10^{-3}$
4. Show:
- convergence iterations,
 - runtime,
 - final delta/error.

3. Policy Visualisation – 1 mark

Visualise: drone movement directions, rescue sequence, preferred charging behaviour, avoidance of dangerous zones.

Students may use: arrows, heat maps, grid overlays, or animated trajectories.

4. State-Value Analysis – 1 mark

Choose a meaningful slice of the state space.

For example:

- Fix rescue target locations and battery level,
- Vary only drone position,
- Plot heatmap of $V^*(s)$.

Students must explain observed patterns.

5. DP Scalability Discussion – 1 mark

Explain: Curse of Dimensionality

How state space changes if:

- grid becomes 10×10 ,
- more rescue targets are added,
- weather conditions become dynamic.

Is DP sufficient? Discuss:

- why DP becomes difficult,
- how Deep RL methods could help,
- relation to real-world autonomous drone systems.

Important : Assignment must be executed in virtual lab and screenshots with timestamp in virtual lab must be attached in the final submission. Also, the timestamp of the execution, Virtual Machine ID shall be fetched & printed in the top of the Python notebook.